

CST334 - Introduction to Operating Systems

CST334 (Operating Systems) Syllabus

Summary

The goal of this course is to introduce you to the design and usage of operating systems. By the end of it, the goal is for you to be able to think about how to design core parts of the operating system, as well as be able to explain why design decisions were made along the way, and know how to navigate and develop software in a systems-oriented manner.

- **Grading:** In addition to earning a passing overall grade, you must score at least 40% in **each** of the three assignment groups. They are Programming Assignments (35%), Exams (35%), and Participation (30%). Final grades are calculated by rounding to the nearest whole number and converted to letter grades using the standard range. *Details can be found in grading.*
- **Late work:** Late submissions are accepted only for Programming assignments and Labs, and have a 10% deduction per calendar day late. *Details can be found in late policy.*
- **Getting help:** There are a ton of office hours spread throughout the week so make use of them if you have any questions! In general, you can use the class slack channel to ask questions (but not post code!), or send me questions (code okay!) via slack or my email. You are **not** allowed to use LLMs or classmates to write code for you, but you can ask them clarifying questions about material. *Details can be found in personnel.*

Course Information

Course Details

- **Course Title:** Introduction to Operating Systems
- **Course Number:** CST334
- **Prerequisites:** CST237, CST 238, and MATH 130

Course Description

A computer's operating system provides convenient features for users and application developers. A power user must know how to use these features and how the operating system provides them. In this course, students will learn about the use and design of modern operating systems, focusing on Linux. On the “use” side, students will learn to navigate the Linux file system, write shell scripts, and combine commands with pipes. Students will also learn to build programs using important GNU utilities such as awk and make. On the “design” side, students will become familiar with core OS design problems, such as how to run multiple applications at once and approaches to designing computer systems.

Course Objectives

At the end of this class, you should be able to:

1. explain core OS design problems and solutions to each of them,
2. write C code that:

1. Demonstrates key operating systems concepts
2. correctly leverages concurrency
3. parses BNF grammars
3. use the Linux command line (e.g. BASH),
4. do command-line scripting,

Topics Covered

The major topics covered in class are:

1. **Using Linux**
2. **Process Management** – How processes are started, stopped, and scheduled.
3. **Memory Management** – How memory is divided up among various processes.
4. **Concurrency** – How we can achieve high-performance sharing of memory.
5. **IO and Persistence** – How we interface with external components.
6. **Language Parsing** – How compilers and interpreters work.

Personnel

- Dr. Sam Ogden (instructor)
 - E-mail: sogden@csumb.edu
 - Web: <https://csumb.edu/scd/sogden/>
 - Office: BIT205
 - Office Hours: 2pm-3pm Mondays & 1pm-2pm Wednesdays, or by appointment

Getting Help: Office hours (both mine and the TAs') are available throughout the week - make use of them! You can also use the class communication platform to ask questions (but not post code/answers on public channels!), or send me questions via email.

Communicating online I will use email, Canvas, and Slack for online class communication.

1. **email** is to be used when you need to officially communicate something to me.
 - e.g. "Can you double check this assignment grade for me?"
2. **Slack** will be used to answer content questions and give quick updates.
 - e.g. "I think I found a bug in this quiz, could you check it out?"
 - We have a slack channel named `#cst334-spring2026` on the CS-U-Monterey workspace that you should join.
3. **Canvas** will have information on assignments and due dates
 - In general I don't read things in canvas messages or comments. For questions or comments please use Slack or email instead.

Materials

- Course textbook available for free online: Operating Systems: Three Easy Pieces
- Course lecture recordings: CST334 on Panopto (Note: Previous semesters videos are great for extra reference but also might change so coming to class is incredibly important!)
- Canvas Course Page: CST334 on Canvas
- Syllabus: CST334 Syllabus

Grading

There are three groups of assignments, briefly described in Table 1. **To pass this course, you must meet both of the following requirements: (1) achieve an overall passing grade (60%), and (2) score at least 40% in each of the three assignment groups below. Failing to meet either requirement will result in failing the course.**

Assignment Kind	Value
Programming assignments	35%
Exams	35%
Participation	30%

Table 1. Points available per assignment type.

Programming Assignments

Programming assignments are bi-weekly assignments in the C programming language that are intended to deepen your understanding of how operating systems work and expose you to the C programming language. In total there are 6 programming assignments, approximately one every two weeks. Programming assignments have submissions due on **Sunday nights**.

For each assignment you will submit `student_code.c` and, optionally, `student_code.h`. You can submit assignments as many times as you like, although only your last submission will be used for your final grade. Late assignments will be accepted, with details in the late policy on submissions.

Each assignment consists of starter code, a README file outlining the assignment, and unit tests. In addition to the supplied unit tests, more comprehensive unit tests may be run by the instructor.

Although the academic honor code is covered in more detail in a later section, all code and checkpoint responses submitted should be written solely by the submitter. Violations of this are considered academic dishonesty. Please consult the section on academic honesty for more details.

Exams

Throughout the semester there will be 4 exams spaced approximately a month apart. Each exam will be a mix of new and old material. Details on which exam first includes prior material are on the course calendar. These exams are in-class and you will have the full class period (110 minutes) to complete them. Exams are paper-based and you may bring in single sheet of notes (handwritten).

Participation

In-class participation has four components:

1. **In-class quizzes** (10%)
2. **Lab completion** (5%)
3. **Weekly Study Notes** (5%)
4. **Research Project** (10%)

In-class quizzes Attendance will be taken through short in-class quizzes or surveys. These surveys may either be based on readings from OSTEP (the appropriate chapters for a day can be found in the class calendar) or be surveys used to check attendance. Quizzes are open-book and open-note and will be completed on canvas.

While these quizzes cannot be made up, the lowest 4 quizzes will be dropped. Please note that these quizzes may not occur every class, and are more likely on days when attendance is low, as a way to encourage attendance.

Labs Throughout the course there will be several labs that are intended to be completed in class. These are designed to help you better understand key topics, get practice with calculations, and get experience with the working environments. Generally, these are designed to take 30 minutes and will happen every two weeks, with submissions due on **Sunday night**. Students are strongly encouraged to work in groups on the labs and use any available outside resources. Although in general I aim to provide sufficient information

to complete the labs, students are encouraged to use any additional resources they find helpful to complete them since they often are only an introduction to the material.

Weekly Study Notes Each week students complete a Weekly Study Note to consolidate and organize their understanding of that week’s material. These notes serve as your personalized study guide throughout the semester—a cumulative resource you’ll build week by week and return to when preparing for exams.

Weekly Study Notes should: - Capture key concepts, techniques, and insights from the week - Note connections between topics and identify areas needing review - Be written in a way that will be useful to your future self when studying

Think of each note as adding another section to your exam review sheet. The clearer and more organized your notes are now, the more valuable they’ll be later when you’re studying.

Research Project During the semester you will complete a group research project. This research project will consist of exploring academic literature related to class and summarizing it, drawing particular attention to how it connects to the course material. Details will be provided during the semester, but it will be a group-based project with both group and individual deliverables.

Extra Credit Potential

Bug Hunting We all make mistakes – my github commit history is proof of that. Sometimes you catch big ones before I do, and if you let me know I might throw some extra credit your way. This is generally reserved for big things (e.g. I left out a bunch of files in the repo or some tools out of the docker image), but if you find yourself confused by something please reach out, maybe you found a novel bug! ¹

Grade Assignment

Grades as assigned based on the standard ranges, which are outlined below, with exceptions made based on the general grading requirements guidance. Note that $[90, 93)$ means “at least 90 but less than 93” (e.g. $90 \leq score < 93$) and that all grades will be rounded to the nearest whole number (e.g. a 92.1 will be rounded to a 92, but a 92.5 will be rounded to a 93). Additionally, note that there will be no curve.

Grade Range	Letter Grade
$[97, \infty)$	A+
$[93, 97)$	A
$[90, 93)$	A-
$[87, 90)$	B+
$[83, 87)$	B
$[80, 83)$	B-
$[77, 80)$	C+
$[73, 77)$	C
$[70, 73)$	C-
$[60, 70)$	D
$[0, 60)$	F

¹I should note that it’s not like there’s a place for direct extra credit for bugs, but I generally find a way to tweak things a bit depending on how big a bug it is. Regardless I always appreciate feedback.

Course Policies

Attendance

Attending lecture is strongly correlated with academic achievement², and thus is strongly encouraged. Attendance is tracked through in-class activities including, but not limited to, reading quizzes.

If you are sick, please do not come to class.

A number of attendance quizzes are dropped to accommodate things that come up in life.

Late Policy

With two exceptions, no late work will be accepted and no extensions will be granted. These two exceptions are programming assignments and lab assignments, both of which can be submitted with a late penalty at any point before the Sunday prior to final exam week. This late penalty will be a 10% reduction in maximum points per day, with a maximum reduction of 60%³.

If you do fall behind in class, please set up a time to meet with me to discuss getting you back on track – my goal is to have you succeed in class and I want to find a way to make that happen.

Academic Honor Code

You may talk to other students or LLMs for:

- Clarification on topics covered in class
- Clarification of what *provided* code is doing
- Generating more examples
- Understanding compiler errors

You may *not* talk to other students or LLMs for:

- Code and assignment answers
- Exam questions and answers

The use of LLMs

At the highest level, the use of LLMs in this class is ***prohibited*** for completing programming assignments and ***encouraged*** for improving your own understanding of material. Large Language Models (LLMs), such as ChatGPT and GPT4.0, are powerful language generation models. They can be used to produce code and summarize text, as well as answer clarifying questions. They can be a very effective tool in a programmer's toolbox if used appropriately. In this class, you should use them as a resource similar to stackoverflow – a good resource that should be critically considered since the code might be wrong, and not as effective as coming to talk to a TA or instructor.

You may use LLMs for asking clarifying questions and generating examples. I will demonstrate a number of these such questions in class but “how do I use XX command?” and “how does a free list in memory management work?” are examples. Additionally, asking for example problems (e.g. “can I have five examples of turnaround time calculation with FIFO and RR scheduling?”) is an excellent use⁴. Further, if you are confused about what a homework problem is asking or how it relates to operating systems at a larger scale you can ask them⁵.

²Attendance and grade are strongly correlated (not just because of the attendance quizzes but they also factor in).

³This is done automatically through canvas and it “chops off” the points that you can't get. For example, if you turn in an assignment 1 day late and would have gotten a 95% on it you will get a 90% for it.

⁴But don't trust their math. They're getting better but not great. Practice quizzes are a better place to check this out as I try to provide walk throughs.

⁵Or, you know, come talk to me. I know a lot more about the specifics of this class than the AI does.

You may *not* use LLMs for producing answers on programming assignments, quizzes or exams. Asking for an LLM's solution to a coding assignment is conceptually the same as asking another student to write you code for you, and will be treated as such. While you may ask for individual parts to help understand the C language better (e.g. "how do I write a do-while loop in C"), using it for more than ~2 lines of code is not allowed. As a rule of thumb, a prompt of "write me a function to do..." indicates that you are headed in a problematic direction.

You are encouraged to use LLMs like you would a TA or an instructor. You *may not* use LLMs for producing code for programming assignments, but *may* use it to learn how to better use the language and to clarify topics. They can clarify complex topics we learned in class, give you alternative explanations and can be incredibly helpful in understanding errors and problems. However, it is important to be able to identify *good* examples and *bad* examples of LLM output.

University Academic Integrity Policy

For complete academic integrity policies, please refer to the CSUMB Academic Integrity Policy.

University Policies and Resources

Enrollment and Registration

For information about requesting an incomplete or withdrawal from the course, please refer to CSUMB's Enrollment and Registration Policy.

For grade appeals, consult the Grade Appeal Policy.

Disability Services

If you have a disability that may require academic accommodations, please contact the Student Disability Resources office as soon as possible. All discussions will remain confidential. Students who have already received approval for accommodations from SDR should schedule a meeting with the instructor as early in the semester as possible.

Collection of Student Work

Student work may be collected and used for course assessment and improvement purposes. By enrolling in this course, you consent to the collection and analysis of your academic work for educational assessment. All student work will be handled confidentially and in accordance with FERPA regulations.

Syllabus Change Policy

This syllabus is subject to change at the instructor's discretion to enhance learning or accommodate unforeseen circumstances. Students will be notified of any substantive changes (such as changes to due dates, point values of assignments, or major policy modifications) via Canvas announcement and/or email at least one week in advance when possible.

The process for syllabus changes: 1. Instructor identifies need for change 2. Revised syllabus section is prepared 3. Students are notified via Canvas and email 4. Change takes effect after notification period

Students are responsible for staying current with any announced changes to the syllabus.